populated with the details of the file upload:

```
app.MapGet("/upload", (IFormFile file) => {});
```

You can also use an `IFormFileCollection` if you need to accept multiple files:

```
app.MapGet("/upload", (IFormFileCollection files) =>
{
    foreach (IFormFile file in files)
    {
    }
});
```

The `IFormFile` object exposes several properties and utility methods for reading the contents of the uploaded file, some of which are shown here:

```
public interface IFormFile
{
    string ContentType { get; }
    long Length { get; }
    string FileName { get; }
    Stream OpenReadStream();
}
```

As you can see, this interface exposes a `FileName` property, which returns the filename that the file was uploaded with. But you know not to trust users, right? You should *never* use the filename directly in your code; users can use it to attack your website and access files that they shouldn't. Always generate a new name for the file before you save it anywhere.

WARNING There are lots of potential threats to consider when accepting file uploads from users. For more information, see http://mng.bz/yQ9q.

The `IFormFile` approach is fine if users are going to be uploading only small files. When your method accepts an `IFormFile` instance, the